

Architecture & Service Flows

How every component fits together, end to end

This document maps the complete runtime architecture of the JSD TATA Emergency Services platform — every AWS service, the trust boundaries between them, and a step-by-step walkthrough of each operational flow (booking, dispatch, ETA, tracking, notifications, AI agents and analytics).

Document	Architecture & Service Flows
Date	25 June 2026
Audience	Executive summary + engineering detail (layered)

Status

Confidential — internal

1. Executive summary

JSD TATA Emergency Services is a serverless emergency dispatch platform for Tata Steel, Jamshedpur. It coordinates ambulances, fire trucks and blood-bank logistics across the township. Citizens and staff request help through a web portal (Cognito SSO); hospital and blood-bank systems integrate machine-to-machine via scoped API keys; dispatchers manage everything from an admin console.

The system is fully serverless: a React single-page app served via CloudFront, a single AWS Lambda behind an API Gateway HTTP API, and DynamoDB for storage. Amazon Bedrock powers a voice agent and a policy-as-config agent; OSRM provides road routing for traffic-aware ETAs; SES/SNS deliver notifications; Power BI Embedded provides analytics. There are no servers to manage and the design scales on demand.

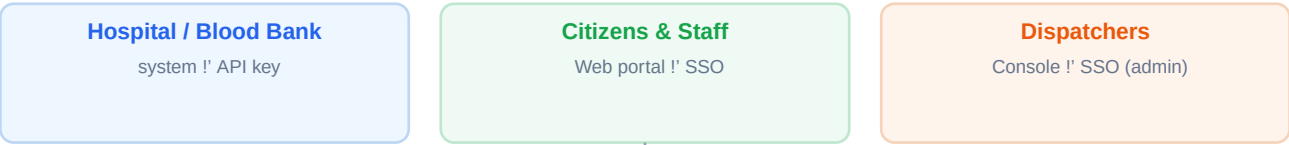
2. Component inventory

Service	Role
Amazon CloudFront + S3	Hosts and globally delivers the React SPA over HTTPS (S3 private via Origin Access Control).
Amazon Cognito	Single sign-on; issues JWTs (RS256) verified in-Lambda against the pool JWKS.
API Gateway (HTTP API)	Single proxy route to the Lambda; stage throttling (10 rps, burst 20); locked CORS.
Lambda: psiog-transport-api	Core API: auth, dispatch engine, ETA, live tracking, notifications, policy upload.
Lambda: psiog-policy-sync	Reads a natural-language policy PDF via Bedrock and updates dispatch config.
DynamoDB	Four tables: ReferenceData, Fleet, TransportRequests, ShuttleCards.
Amazon Bedrock (Nova Lite)	Voice-agent intent extraction and policy-document understanding.
OSRM	Road-network routing for distance/time, scaled by a traffic factor.
Amazon SES / SNS	Email and SMS notifications to requesters.
Power BI Embedded	Embedded analytics dashboard (App-owns-data, SSO-gated).

Master architecture

All services and how requests flow from clients to data and external systems.

CLIENTS



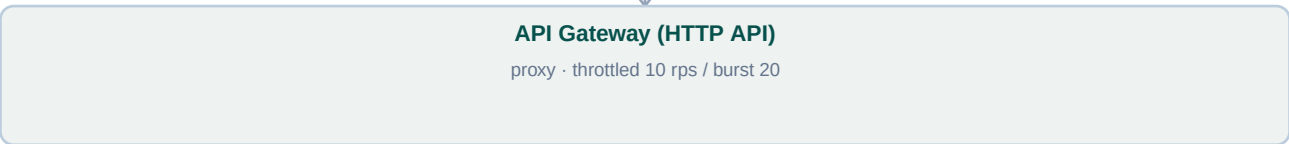
EDGE / DELIVERY



IDENTITY



API



COMPUTE



DATA (DYNAMODB)



EXTERNAL / AI / MESSAGING



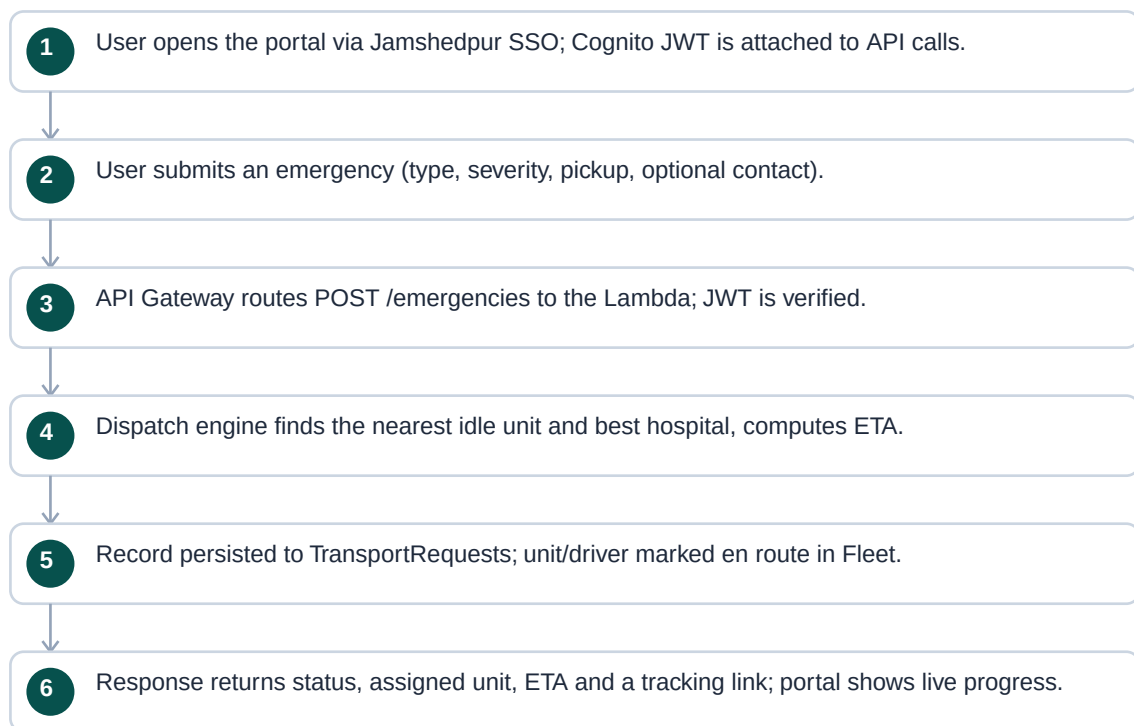
3. Trust boundaries & authentication

Two principal types are supported. Browser users authenticate with a Cognito JWT (Authorization: Bearer); the Lambda verifies the signature against the pool JWKS, checks expiry/issuer/audience, and reads `cognito:groups` for the admin role (`transport-admin`). Server callers (hospital, blood bank) use a scoped `x-api-key`, mapped to a source with a permission scope (`CONSOLE` = all; `HOSPITAL` = emergencies only). All operational reads/writes require one of these; the public live-tracking endpoint is the only unauthenticated route and is protected by a per-incident secret token.

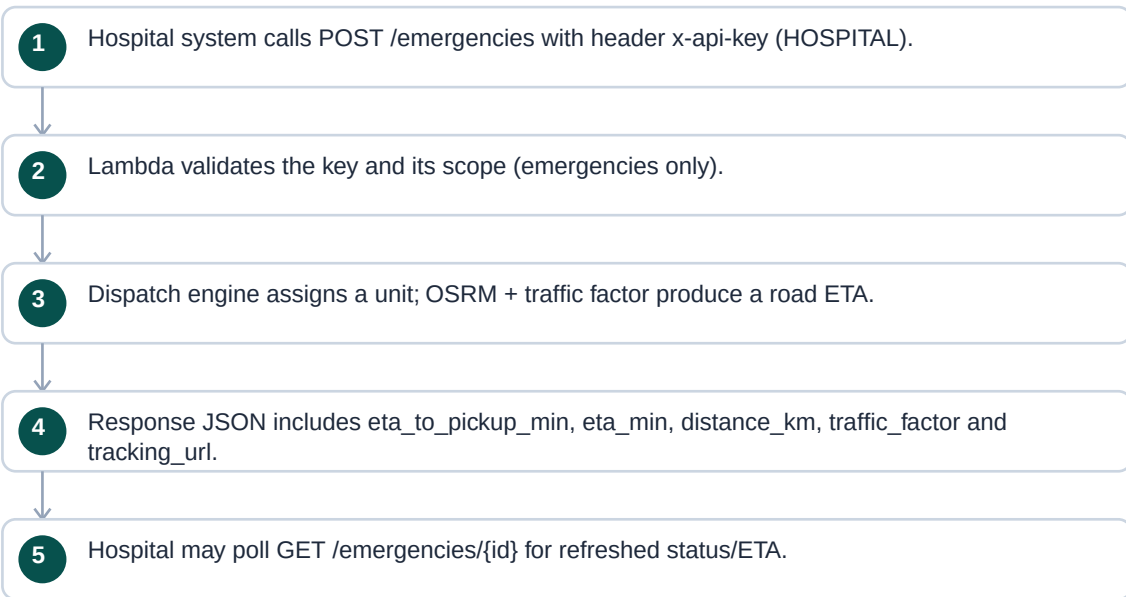
4. End-to-end flows

Each flow below lists the runtime steps. All write paths pass through the dispatch engine in the core Lambda, which assigns the nearest available unit and persists the result to DynamoDB.

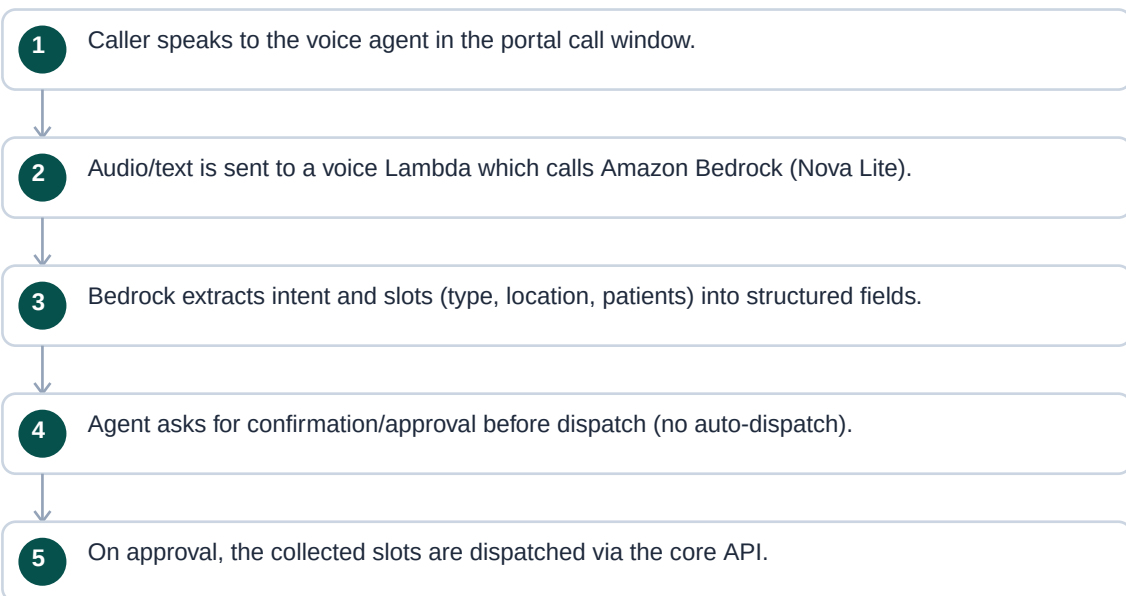
4.1 Citizen / staff booking (web portal)



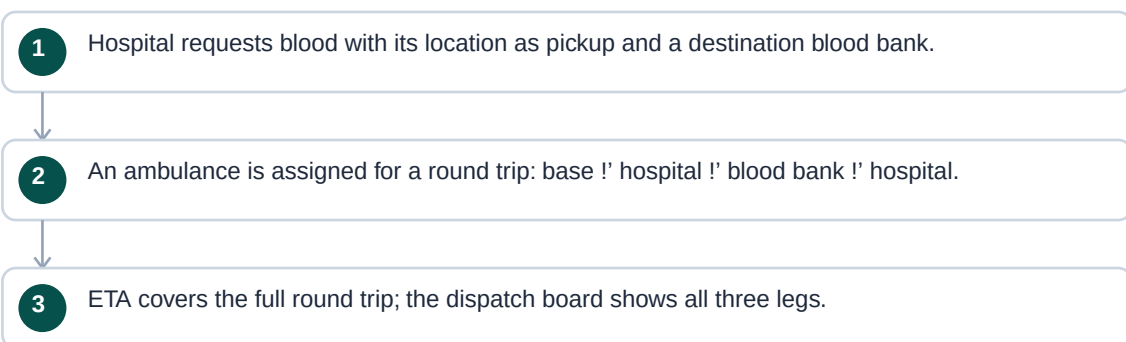
4.2 Hospital / blood-bank booking (API key)



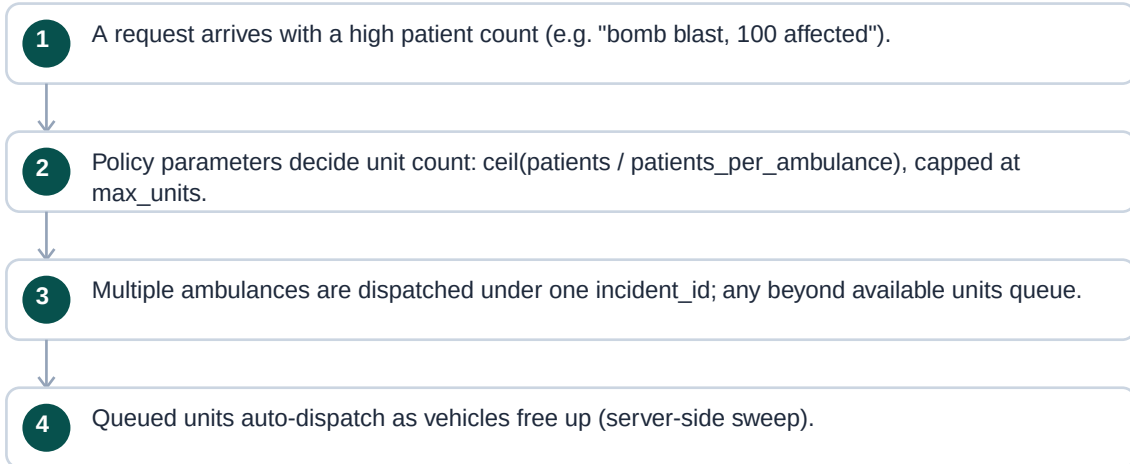
4.3 Voice-agent booking



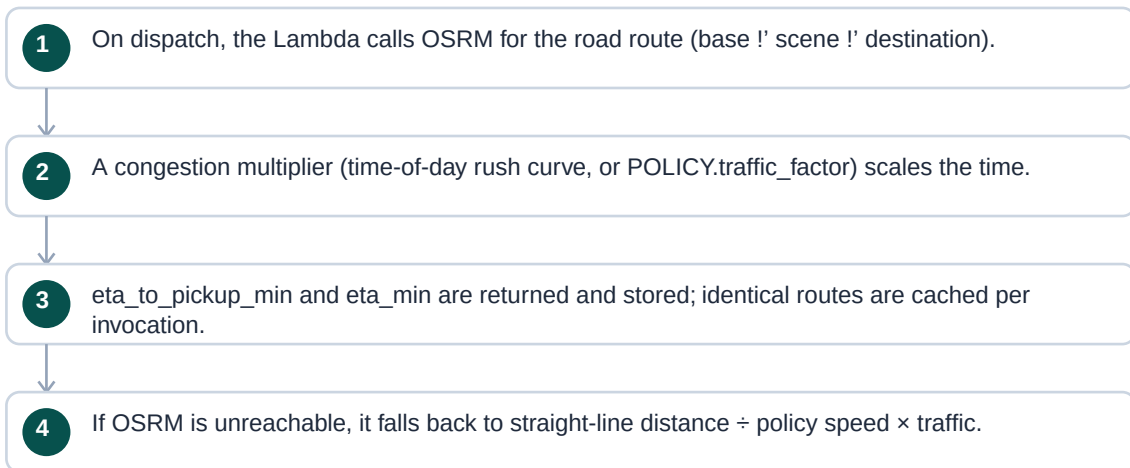
4.4 Blood-bank logistics (round trip)



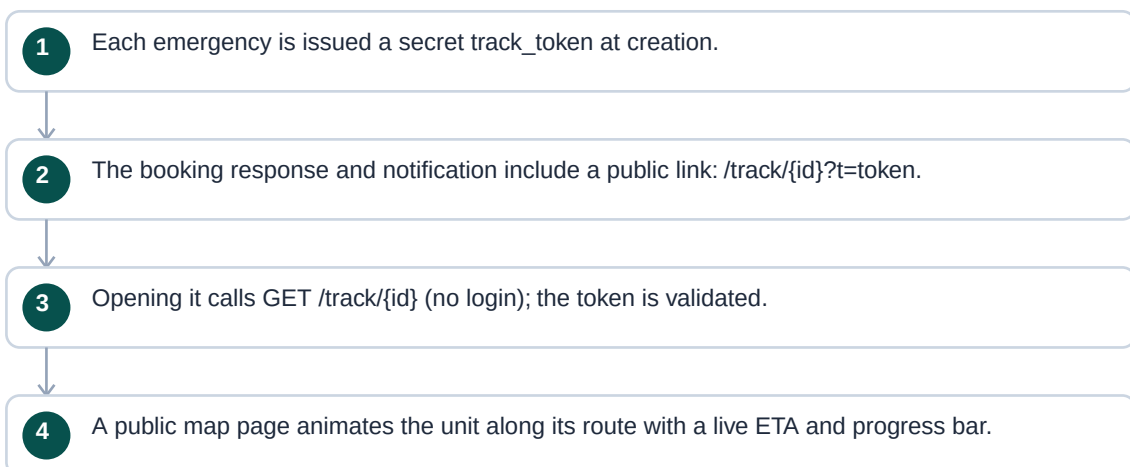
4.5 Mass-casualty dispatch



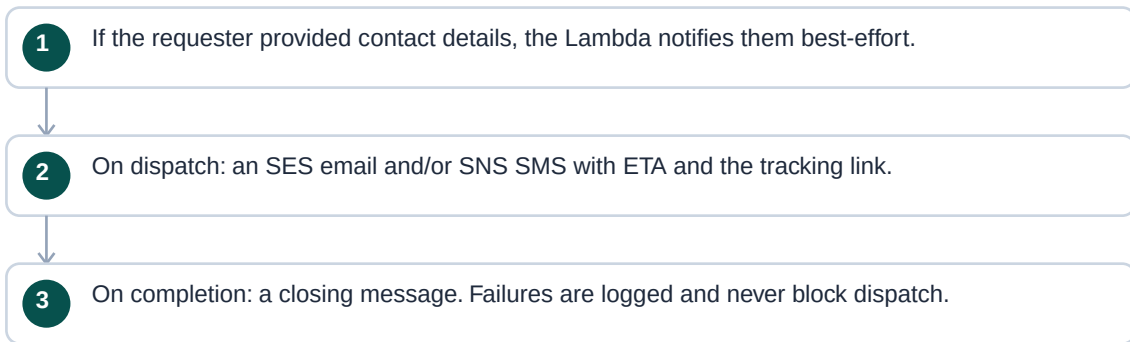
4.6 Real-time traffic & ETA



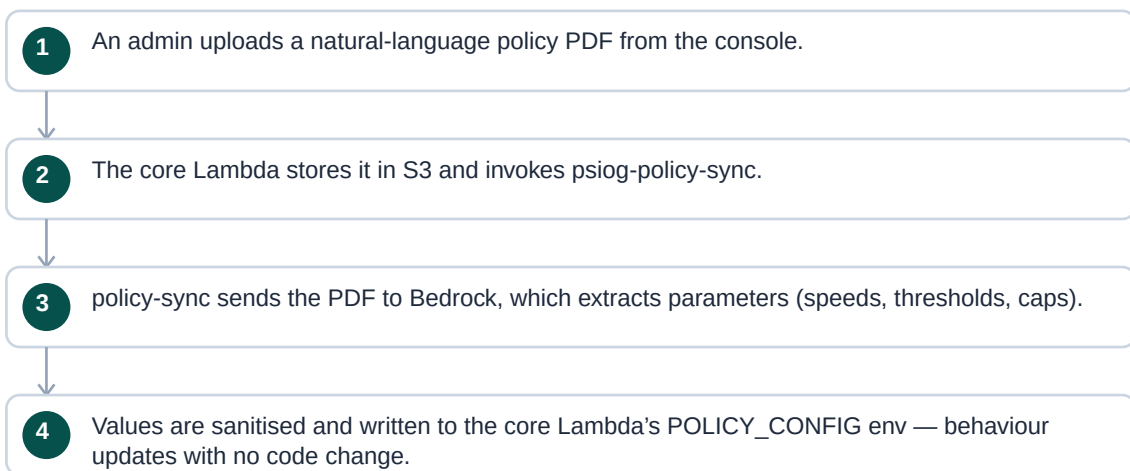
4.7 Live tracking link



4.8 Notifications (email / SMS)



4.9 Policy-as-config agent



4.10 Power BI analytics (App-owns-data)

